

الدرس 3 – بطاقة مرجعية سريعة

إنشاء المتحكمات ؟

```
# متحكم بسيط
php artisan make:controller ProductController

# Resource Controller
php artisan make:controller ProductController --resource

# API Resource Controller
php artisan make:controller API/ProductController --api

# مع Model
php artisan make:controller ProductController --resource --model=Product

# Single Action Controller
php artisan make:controller ShowProfileController --invokable
```

أساسي Controller هيكل ؟

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Product;

class ProductController extends Controller
{
    public function index()
    {
        $products = Product::all();
        return view('products.index', compact('products'));
    }

    public function show($id)
    {
        $product = Product::findOrFail($id);
        return view('products.show', compact('product'));
    }
}
```

```
}  
}
```

❓ مع المسارات Controllers ربط

```
// مسار بسيط  
Route::get('/products', [ProductController::class, 'index']);  
  
// مع parameter  
Route::get('/products/{id}', [ProductController::class, 'show']);  
  
// Resource Routes (7 مسارات تلقائياً)  
Route::resource('products', ProductController::class);  
  
// API Resource (5 مسارات)  
Route::apiResource('products', ProductController::class);  
  
// Single Action Controller  
Route::get('/profile/{id}', ShowProfileController::class);
```

❑ Resource Controller Methods

```
class ProductController extends Controller  
{  
    // GET /products  
    public function index()  
    {  
        return view('products.index');  
    }  
  
    // GET /products/create  
    public function create()  
    {  
        return view('products.create');  
    }  
  
    // POST /products  
    public function store(Request $request)
```

```

{
    Product::create($request->validated());
    return redirect()->route('products.index');
}

// GET /products/{id}
public function show(Product $product)
{
    return view('products.show', compact('product'));
}

// GET /products/{id}/edit
public function edit(Product $product)
{
    return view('products.edit', compact('product'));
}

// PUT/PATCH /products/{id}
public function update(Request $request, Product $product)
{
    $product->update($request->validated());
    return redirect()->route('products.show', $product);
}

// DELETE /products/{id}
public function destroy(Product $product)
{
    $product->delete();
    return redirect()->route('products.index');
}
}

```

حقن التبعية (Dependency Injection) ؟

```

// حقن Request
public function store(Request $request)
{
    $name = $request->input('name');
    $price = $request->input('price');
}

// حقن Model (Route Model Binding)
public function show(Product $product)

```

```

{
    // $product جاهز للاستخدام
    return view('products.show', compact('product'));
}

// حقن Service
public function store(Request $request, ProductService $service)
{
    $product = $service->createProduct($request->all());
    return redirect()->route('products.show', $product);
}

// حقن في Constructor
class ProductController extends Controller
{
    protected $service;

    public function __construct(ProductService $service)
    {
        $this->service = $service;
    }

    public function index()
    {
        $products = $this->service->getAllProducts();
        return view('products.index', compact('products'));
    }
}

```

□ Middleware في Controllers

```

// في Constructor
class ProductController extends Controller
{
    public function __construct()
    {
        // كل methods
        $this->middleware('auth');

        // index و show ماعدا
        $this->middleware('auth')->except(['index', 'show']);

        // فقط create و store
    }
}

```

```
        $this->middleware('auth')->only(['create', 'store']);
    }
}
```

// في المسارات

```
Route::resource('products', ProductController::class)
    ->middleware('auth');
```

معيّنة Methods تحديد ؟؟

// فقط index و show

```
Route::resource('products', ProductController::class)
    ->only(['index', 'show']);
```

// كل شيء ماعدا destroy

```
Route::resource('products', ProductController::class)
    ->except(['destroy']);
```

أنواع Responses ؟

```
// View
return view('products.index', compact('products'));
```

```
// Redirect
return redirect()->route('products.index');
```

// Redirect مع رسالة

```
return redirect()->route('products.index')
    ->with('success', 'تم بنجاح');
```

// JSON (للـ APIs)

```
return response()->json([
    'status' => 'success',
    'data' => $products
]);
```

```
// Download
```

```
return response()->download($filePath);
```

التحقق من البيانات ؟

```
// في Controller
public function store(Request $request)
{
    $validated = $request->validate([
        'name' => 'required|max:255',
        'price' => 'required|numeric|min:0',
        'email' => 'required|email|unique:users',
    ]);

    Product::create($validated);
}

// باستخدام Form Request
public function store(StoreProductRequest $request)
{
    // التحقق يتم تلقائياً
    Product::create($request->validated());
}
```

إنشاء Form Request:

```
php artisan make:request StoreProductRequest
```

Single Action Controller

```
class ShowProfileController extends Controller
{
    public function __invoke($id)
    {
        $user = User::findOrFail($id);
        return view('profile', compact('user'));
    }
}
```

// الاستخدام

```
Route::get('/profile/{id}', ShowProfileController::class);
```

❓ أوامر مفيدة

```
# عرض جميع المسارات
php artisan route:list

# معين controller مسارات
php artisan route:list --name=products

# إنشاء controller
php artisan make:controller ProductController --resource

# إنشاء Form Request
php artisan make:request StoreProductRequest

# عرض Resource Routes بنية
php artisan route:list --path=products
```

❓ أفضل الممارسات

❑ **CRUD** للـ **Resource Controllers** استخدم

```
Route::resource('products', ProductController::class);
```

❑ **Route Model Binding** استخدم

```
public function show(Product $product) { }
```

❑ **Form Requests** استخدم

```
public function store(StoreProductRequest $request) { }
```

❑ **Controller** لا تضع منطق معقد في

```
// استخدم Service Classes  
public function store(Request $request, ProductService $service) { }
```

استخدم أسماء واضحة □

```
class ProductController extends Controller  
class UserProfileController extends Controller
```

روابط سريعة ؟ □

- الدرس الرئيسي
- الدرس التالي